# PostgreSQL Replication Overview

## Table of Contents



---

## Learning Objectives

By the end of this module, you will be able to:

- Explain the difference between physical and logical replication
- Choose the right replication type for different use cases
- Understand synchronous vs asynchronous trade-offs
- Describe how WAL (Write-Ahead Log) underpins PostgreSQL replication
- Identify replication lag and its implications
- Design a basic replication topology for production use

---

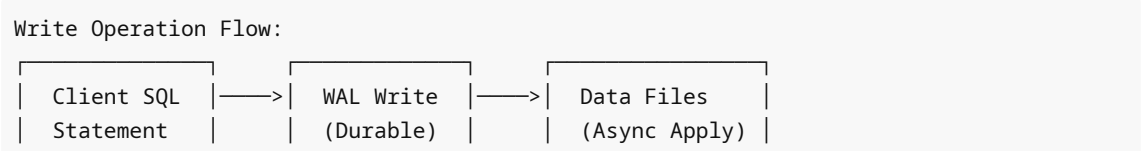
## Introduction to Replication

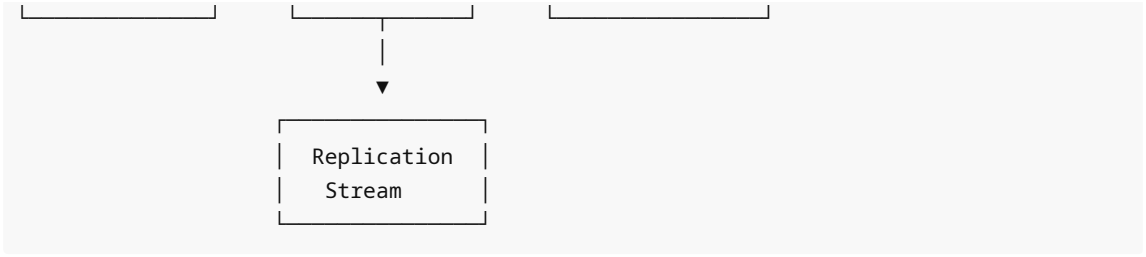
Replication in PostgreSQL is the process of copying data from one database server (primary) to one or more database servers (standbys/replicas). It serves several critical purposes:

- **High Availability (HA):** Automatic or manual failover when the primary fails
- **Read Scalability:** Distribute read queries across multiple replicas
- **Disaster Recovery:** Maintain geographically distributed copies
- **Zero-Downtime Migrations:** Logical replication enables schema changes and major version upgrades
- **Data Distribution:** Replicate subsets of data to different systems

### WAL: The Foundation of Replication

All PostgreSQL replication is built on the **Write-Ahead Log (WAL)**. Every change to the database is first written to the WAL before being applied to data files. This ensures durability and provides the stream of changes needed for replication.






---

## Physical vs Logical Replication

### Physical Replication (Streaming Replication)

Physical replication copies the **exact binary content** of the database cluster at the block level. The standby is a byte-for-byte copy of the primary.

**How it works:**

1. Primary generates WAL records (binary block changes)
2. WAL sender process streams records to standby
3. WAL receiver on standby writes them to local WAL
4. Startup process replays WAL to keep data files current

```
-- View physical replication status on primary
SELECT client_addr, state, sent_lsn, write_lsn, flush_lsn, replay_lsn,
       write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

**Characteristics:**

| Property | Value |
|---|---|
| Granularity | Block-level (entire cluster) |
| Standby state | Read-only (hot standby) or no connections |
| Cross-version | No (same major version required) |
| Cross-platform | Limited (same architecture) |
| Setup complexity | Low |
| Overhead | Low |

### Logical Replication

Logical replication copies data at the **row/tuple level** using a logical decoding of WAL records. It replicates logical changes (INSERT, UPDATE, DELETE) rather than physical blocks.

**How it works:**

1. Primary decodes WAL into logical change records
2. Publisher sends changes for subscribed tables
3. Subscriber applies changes as DML statements

```
-- Create a publication on primary
CREATE PUBLICATION my_pub FOR TABLE orders, customers;

-- Create a subscription on replica
CREATE SUBSCRIPTION my_sub
  CONNECTION 'host=primary port=5432 dbname=mydb user=replicator'
  PUBLICATION my_pub;

-- Monitor logical replication
SELECT subname, received_lsn, latest_end_lsn, latest_end_time
FROM pg_stat_subscription;
```

**Characteristics:**

| Property | Value |
|---|---|
| Granularity | Table/row level |
| Standby state | Full read-write access |
| Cross-version | Yes (PostgreSQL 10+) |
| Cross-platform | Yes |
| Setup complexity | Medium |
| Overhead | Higher (decoding cost) |

### Side-by-Side Comparison

```
PHYSICAL REPLICATION                LOGICAL REPLICATION
─────────────────────────────────   ──────────────────────────────────

Primary ──WAL blocks──> Standby     Primary ──row changes──> Subscriber

✓ Simple setup                      ✓ Table-level granularity
✓ Low overhead                      ✓ Cross-version upgrades
✓ Exact copy (DR ready)             ✓ Bidirectional possible
✗ Must be same PG version           ✗ Sequences not replicated
✗ Replicates entire cluster         ✗ DDL not replicated
✗ Cannot write to standby           ✗ Higher CPU overhead
```

---

## Synchronous vs Asynchronous Replication

### Asynchronous Replication (Default)

The primary commits transactions without waiting for the standby to acknowledge receipt or apply the WAL.

```
Async Flow:
Client ──COMMIT──> Primary ──ACK──> Client
                      │
                      └──WAL──> Standby (eventually)
```

```
Risk: If primary crashes before standby receives WAL,
      data loss up to replication lag amount.
```

```
# postgresql.conf - Async (default)
synchronous_standby_names = ''   # empty = async
wal_level = replica
max_wal_senders = 10
```

### Synchronous Replication

The primary waits for one or more standbys to confirm WAL receipt before acknowledging COMMIT to the client.

```
Sync Flow:
Client ──COMMIT──> Primary ──WAL──> Standby
                      │                │
                      └────────ACK─────┘
                      │
                  ACK to Client

Guarantee: Zero data loss (RPO=0) when standby is available.
```

```
# postgresql.conf - Synchronous
synchronous_standby_names = 'standby1'
# Or with priority groups:
synchronous_standby_names = 'FIRST 1 (standby1, standby2)'
# Or quorum commit:
synchronous_standby_names = 'ANY 1 (standby1, standby2, standby3)'
```

### Synchronous Commit Levels

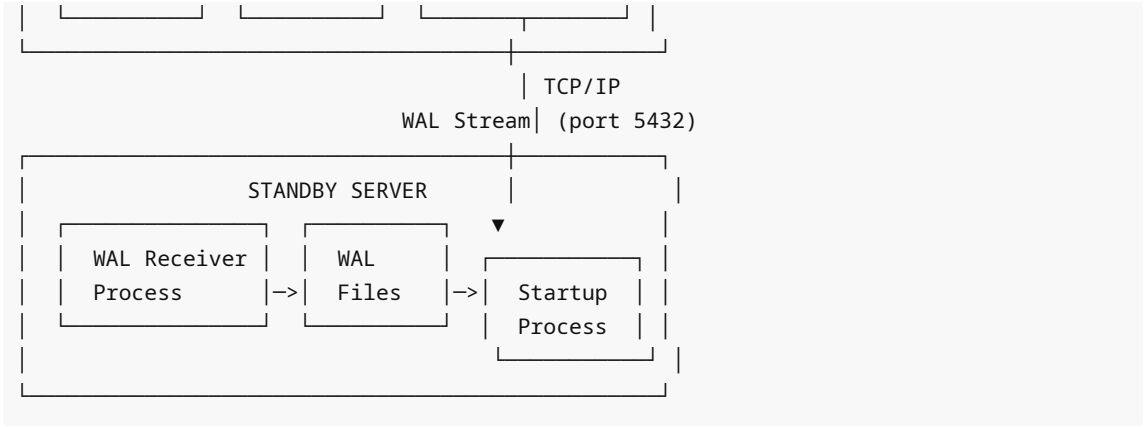
```
-- Per-transaction synchronous commit control
SET synchronous_commit = 'on';          -- Wait for flush on sync standby
SET synchronous_commit = 'remote_apply';-- Wait for apply on standby
SET synchronous_commit = 'remote_write';-- Wait for OS write on standby
SET synchronous_commit = 'local';       -- Wait for local flush only
SET synchronous_commit = 'off';         -- No wait (async, risk 1 WAL buffer loss)
```

---

## Replication Architecture Diagrams

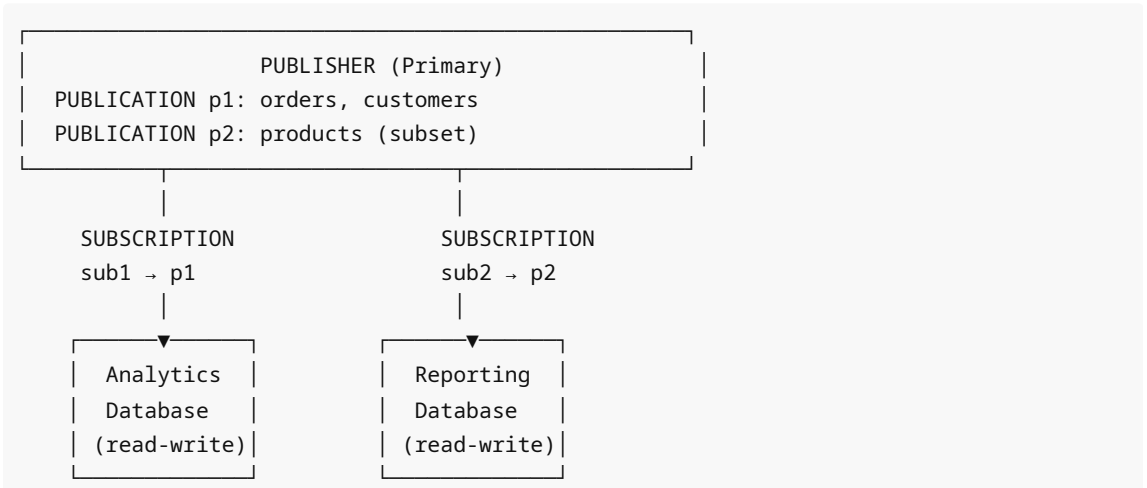
### Basic Primary-Standby

```
┌──────────────────────────────────────────────────┐
│                  PRIMARY SERVER                  │
│  ┌────────────┐  ┌────────────┐  ┌──────────────┐ │
│  │ Postgres   │  │  WAL       │  │  WAL Sender  │ │
│  │ Process    │─>│  Writer    │─>│  Process     │ │
```




### Cascading Replication


### Logical Replication Multi-Subscriber




---

## Use Cases and Selection Guide

### Decision Matrix

```
│
├── Major version upgrade (zero downtime)?
│   └── Use LOGICAL Replication
│
├── Replicate subset of tables?
│   └── Use LOGICAL Replication
│
├── Bidirectional replication?
│   └── Use LOGICAL Replication (carefully!)
│
├── RPO = 0 (zero data loss)?
│   └── Use SYNCHRONOUS Physical Replication
│
└── Cross-database/cross-system replication?
    └── Use LOGICAL Replication
```

### Common Use Case Scenarios

| Scenario | Recommended Approach |
|---|---|
| Primary/standby HA | Streaming + Patroni |
| Read replicas | Async streaming replication |
| Zero-downtime major upgrade | Logical replication migration |
| Multi-region DR | Async streaming (accept small lag) |
| OLAP reporting copy | Logical replication to analytics DB |
| Zero data loss financial | Synchronous streaming |
| ETL/CDC pipeline | Logical replication + pgoutput |

---

## Configuration Overview

### Primary Server (postgresql.conf)

```
# Replication basics
wal_level = replica              # 'replica' for streaming, 'logical' for logical
max_wal_senders = 10             # Max concurrent WAL sender processes
max_replication_slots = 10       # Max replication slots
wal_keep_size = 1GB              # Keep WAL for standbys without slots

# Performance
wal_compression = on             # Compress WAL (saves bandwidth)
wal_log_hints = on               # Needed for pg_rewind

# Monitoring
track_commit_timestamp = on      # Track when each transaction was committed
```

### Primary Server (pg_hba.conf)

```
# Allow replication connections
# TYPE  DATABASE        USER            ADDRESS         METHOD
host    replication     replicator      10.0.0.0/8      scram-sha-256
host    replication     replicator      192.168.1.0/24  scram-sha-256
```

---

## Monitoring Replication

### Key System Views

```sql
-- On PRIMARY: replication status
SELECT
    client_addr,
    application_name,
    state,              -- 'streaming', 'catchup', 'backup'
    sent_lsn,
    write_lsn,
    flush_lsn,
    replay_lsn,
    write_lag,
    flush_lag,
    replay_lag,
    sync_state          -- 'async', 'sync', 'potential', 'quorum'
FROM pg_stat_replication;

-- On STANDBY: receiver status
SELECT
    status,
    receive_start_lsn,
    received_lsn,
    last_msg_send_time,
    last_msg_receipt_time,
    latest_end_lsn,
    latest_end_time
FROM pg_stat_wal_receiver;

-- Check replication lag in seconds
SELECT
    application_name,
    EXTRACT(EPOCH FROM replay_lag) AS lag_seconds
FROM pg_stat_replication;

-- Replication slots
SELECT slot_name, slot_type, active, restart_lsn, confirmed_flush_lsn
FROM pg_replication_slots;
```

### Alerting Thresholds

```sql
-- Alert if replication lag > 30 seconds
SELECT client_addr, replay_lag
FROM pg_stat_replication
WHERE replay_lag > INTERVAL '30 seconds';

-- Alert if replication slot has unconsumed WAL (disk risk)
SELECT slot_name,
       pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn) AS lag_bytes
FROM pg_replication_slots
WHERE active = false;
```

---

## Common Mistakes

1. **Not setting `wal_level = logical`** when you need logical replication — `replica` level is insufficient
2. **Forgetting `max_replication_slots`** — replication slots silently fail to create
3. **Unused replication slots** — they prevent WAL cleanup, causing disk to fill up
4. **Using synchronous replication without understanding** — if standby goes down, primary blocks ALL writes
5. **Not monitoring replication lag** — discovering hours of lag during an incident
6. **Same PostgreSQL version assumption** — physical replication requires exact same major version
7. **No `wal_keep_size`** — standby falls behind and cannot reconnect, needing full re-sync
8. **Replicating to the wrong schema** — logical replication requires matching table schemas

---

## Best Practices

1. Always use **replication slots** for logical replication subscribers
2. Set `wal_keep_size` as a safety net (e.g., 1-2GB) even when using slots
3. Monitor **replication lag continuously** with alerting
4. Test **failover procedures regularly** (at least quarterly)
5. Use `pg_rewind` instead of full re-sync after failover
6. Consider **cascading replication** to offload WAL sender load
7. For synchronous: use `ANY N (list)` quorum mode for better availability
8. Drop unused replication slots immediately

---

## Performance Considerations

| Factor | Physical | Logical |
|---|---|---|
| CPU overhead on primary | Low | Medium-High (decoding) |
| Network bandwidth | Medium (blocks) | Lower (row changes) |
| Standby apply latency | Very Low | Low-Medium |
| Impact on primary vacuums | None | Can delay with inactive slots |
| Hot standby query conflicts | Yes | No (subscriber is read-write) |

---

## Interview Questions

**Q1: What is the difference between physical and logical replication in PostgreSQL?**

A: Physical replication copies data at the binary block level — it streams WAL records that represent exact byte changes to data files. The standby becomes an exact copy. Logical replication decodes WAL into row-level changes (INSERT/UPDATE/DELETE) and sends those to subscribers. Physical is simpler and lower overhead but requires same version/architecture. Logical allows selective table replication, cross-version upgrades, and writable subscribers.

**Q2: What is WAL and why is it central to replication?**

A: WAL (Write-Ahead Log) is PostgreSQL's durability mechanism — every database change is written to WAL before data files are modified. This sequential log is exactly what gets shipped to standbys. Physical replication ships raw WAL blocks; logical replication decodes those WAL records into logical row changes. Without WAL, replication would require copying data files directly.

**Q3: When would you choose synchronous over asynchronous replication?**

A: Synchronous replication is chosen when RPO=0 (zero data loss) is required — typically for financial transactions, medical records, or any system where losing even one committed transaction is unacceptable. The trade-off is increased latency (each commit must wait for standby acknowledgment) and availability risk (if all sync standbys go down, primary blocks writes). Asynchronous is chosen when performance and availability are prioritized over zero data loss.

**Q4: What is a replication slot and what problem does it solve?**

A: A replication slot is a mechanism that ensures the primary retains WAL segments until the subscriber has consumed them. Without slots, if a standby falls too far behind, the primary may delete WAL files the standby needs, causing the standby to disconnect and require full re-sync. Slots guarantee WAL retention but risk disk exhaustion if a subscriber goes offline indefinitely.

**Q5: Can a logical replication subscriber have triggers and foreign keys?**

A: Yes — since logical replication applies changes as regular DML, triggers and foreign key constraints on the subscriber fire normally. This can be both useful (for auditing) and problematic (if triggers cause unintended side effects). Use `ALTER SUBSCRIPTION ... DISABLE TRIGGER` or `session_replication_role = replica` to suppress triggers if needed.

**Q6: What does `synchronous_standby_names = 'ANY 2 (s1, s2, s3)'` mean?**

A: This is quorum-based synchronous replication. The primary waits for ANY 2 out of the 3 listed standbys to confirm WAL receipt before acknowledging a commit. This provides zero data loss while improving availability — if one standby is offline, the other two still form a quorum. Without `ANY`, the `FIRST N` mode requires N standbys in priority order.

**Q7: How does cascading replication work?**

A: A standby can be configured to stream WAL to downstream standbys rather than receiving directly from the primary. This reduces load on the primary (fewer WAL sender connections) and allows replication across network segments. The intermediate standby acts as a relay, and the recovery_min_apply_delay can be set differently per level.

**Q8: What is `wal_level` and what are the valid options?**

A: `wal_level` controls how much information is written to WAL. Options: `minimal` (just enough for crash recovery, no replication), `replica` (sufficient for streaming/physical replication and read-only hot standbys), `logical` (adds information needed for logical decoding/replication). Changing this requires a restart.

**Q9: How do you check replication lag in PostgreSQL?**

A: On the primary, query `pg_stat_replication`: the `replay_lag` column shows the time delay between when a WAL record was generated and when the standby confirmed applying it. You can also compute byte lag with `pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)`. On the standby, `pg_stat_wal_receiver` shows the last received LSN and timestamps.

**Q10: What happens when the synchronous standby goes down?**

A: All transactions on the primary that use synchronous commit will block indefinitely waiting for standby acknowledgment. This is a critical availability concern. Solutions: (1) Use quorum `ANY N` with more standbys than the quorum number. (2) Set `synchronous_commit = local` at the session level for non-critical operations. (3) Configure an automatic timeout with external tooling (Patroni handles this).

**Q11: Can logical replication replicate DDL changes?**

A: No — logical replication in native PostgreSQL does NOT replicate DDL (CREATE TABLE, ALTER TABLE, etc.). Only DML is replicated. For zero-downtime schema changes, you must apply DDL on subscribers before or after changing the publication, depending on the change type. Tools like pglogical or custom event triggers can add DDL replication.

**Q12: What is `hot_standby` parameter and what does it enable?**

A: `hot_standby = on` (default since PostgreSQL 9.0) allows read-only queries on a physical standby while it is in recovery mode. Without it, the standby rejects all connections. With it, clients can connect and run SELECT queries, making standbys useful for read scaling. Read-only queries may be cancelled due to "query conflict with recovery" if they conflict with cleanup operations being replayed.

---

## Exercises

### Exercise 1: Identify Replication Type

Given these requirements, choose physical or logical and justify:

- Need to replicate 3 tables from PostgreSQL 13 to PostgreSQL 16 for upgrade
- Need a hot standby for HA with automatic failover
- Need to stream order changes to an analytics database

**Solution:**

- Cross-version table migration → **Logical** (cross-version, selective)
- HA hot standby → **Physical** (exact copy, low overhead, failover ready)
- Analytics streaming → **Logical** (subscribe only to orders table, read-write analytics DB)

### Exercise 2: Replication Monitoring Query

Write a query that shows all standbys with lag > 10 seconds and whether they are sync or async.

```
SELECT
    application_name,
    client_addr,
```

```
    state,
    sync_state,
    replay_lag,
    pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS lag_bytes
FROM pg_stat_replication
WHERE replay_lag > INTERVAL '10 seconds'
ORDER BY replay_lag DESC;
```

### Exercise 3: Design a Replication Topology

Design replication for: 1 primary, 1 sync standby for HA, 2 async read replicas, 1 DR standby in another region.

```
PRIMARY (sync_standby_names = 'standby1')
   │
   ├──[SYNC]──> STANDBY1 (same DC, HA failover candidate)
   │
   ├──[ASYNC]─> READ_REPLICA1 (same DC, read scaling)
   │
   ├──[ASYNC]─> READ_REPLICA2 (same DC, read scaling)
   │
   └──[ASYNC]─> DR_STANDBY (remote DC, disaster recovery)
```

---

## Cross-References

- 02_streaming_replication.md — Detailed streaming setup
- 03_logical_replication.md — Logical replication deep dive
- 04_synchronous_replication.md — Synchronous configuration
- 05_failover_procedures.md — Failover and promotion
- 09_ha_architecture_patterns.md — Complete HA designs